FD01-Informe-Factibilidad

Logo UPT

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingenieria de Sistemas

Proyecto EPIS Analytics

Curso: *Inteligencia de negocios*

Docente: *Mag. Patrick Cuadros Quiroga*

Integrantes:

Enzo Leonel Laqui Luyo (2022073907)

Steven Christopher Yizuka Baldeón (2002023628)

Tacna - Peru

2026

Sistema EPIS Analytics

Informe de Factibilidad

Version *1.0*

CONTROL DE VERSIONES					
Version	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Equipo del proyecto	Equipo del proyecto	Docente del curso	25/04/2026	Version inicial del informe de factibilidad

Indice General

1. Descripcion del Proyecto
2. Riesgos
3. Analisis de la Situacion Actual
4. Estudio de Factibilidad
5. Analisis Financiero
6. Conclusiones
7. Referencias

Informe de Factibilidad

1. Descripcion del Proyecto

1.1. Nombre del proyecto

EPIS Analytics: Sistema de analisis de contenido social para estudiantes de la Escuela Profesional de Ingenieria de Sistemas.

1.2. Duracion del proyecto

El proyecto se plantea para un periodo academico, considerando las fases de analisis, diseno, implementacion, pruebas, documentacion y presentacion final. Para el prototipo funcional se estima una duracion aproximada de 8 a 10 semanas.

Fase	Duracion estimada	Actividades principales
Analisis y alcance	1 semana	Identificacion del problema, objetivos, usuarios y requerimientos
Diseno	1 semana	Arquitectura cliente-servidor, modelo de datos, flujo de login y dashboard
Implementacion backend	2 semanas	Modelos, repositorios, servicios, controladores REST y autenticacion
Implementacion frontend	2 semanas	Angular, vistas, filtros, graficos y consumo de API
Pruebas y ajustes	1 semana	Pruebas unitarias, pruebas de API, validacion visual y correcciones
Documentacion	1 a 2 semanas	Informes, README, arquitectura, manual tecnico y factibilidad

1.3. Descripcion

EPIS Analytics es un sistema web orientado al analisis de publicaciones e interacciones digitales asociadas a estudiantes de EPIS. El sistema permite visualizar informacion simulada de redes sociales relevantes como LinkedIn, Instagram y YouTube, considerando ciclos academicos del I al X.

El proyecto surge de la necesidad de contar con una herramienta que facilite la observacion del comportamiento digital academico y profesional de los estudiantes. A traves de metricas, graficos y rankings, el sistema ayuda a interpretar el alcance, las interacciones y la participacion por red social, ciclo y estudiante.

La solucion se organiza bajo una arquitectura cliente-servidor. El frontend se implementa con Angular y actua como capa de presentacion. El backend se implementa con FastAPI y expone servicios REST protegidos mediante autenticacion con token. La persistencia se

maneja mediante SQLite para el prototipo y se estructura con repositorios para facilitar una futura migracion a una base de datos gestionada.

1.4. Objetivos

1.4.1. Objetivo general

Desarrollar un sistema web de analisis de contenido social para EPIS que permita consultar metricas de publicaciones, interacciones, alcance y participacion de estudiantes, mediante dashboards filtrables y acceso autenticado.

1.4.2. Objetivos especificos

Objetivo especifico	Resultado esperado
Implementar una arquitectura cliente-servidor con Angular y FastAPI	Separacion clara entre interfaz web, API REST, servicios, repositorios y modelos
Registrar y consultar publicaciones simuladas por estudiante, ciclo y red social	Dataset academico controlado para probar analitica sin depender de APIs externas
Implementar dashboards personal y global	Visualizacion de metricas individuales y agregadas
Incorporar filtros por red social	Consulta general y filtrada para LinkedIn, Instagram y YouTube
Implementar login real basico	Acceso mediante usuario, contrasena, token firmado y rutas protegidas
Realizar pruebas unitarias y de API	Validacion de funcionalidad de modelo, repositorio, servicio y controladores
Elaborar un analisis de costos con enfoque Terraform	Estimacion de infraestructura cloud declarada como codigo y costos mensuales

2. Riesgos

Riesgo	Probabilidad	Impacto	Mitigacion
Cambios en el alcance del proyecto	Media	Alto	Mantener funcionalidades priorizadas: login, dashboard personal, dashboard global y filtros
Datos reales no disponibles por restricciones de redes sociales	Alta	Medio	Usar datos simulados representativos y documentar que el prototipo no consume APIs reales
Complejidad al integrar autenticacion real	Media	Medio	Usar autenticacion local con hash de contrasena y token firmado para el prototipo
Costos cloud no controlados	Media	Alto	Usar Terraform, etiquetas de recursos, estimacion con Infracost y ambiente de bajo consumo
Dependencia de herramientas externas	Media	Medio	Mantener instrucciones de instalacion y comandos reproducibles
Errores de CORS o comunicacion frontend-backend	Media	Medio	Configurar origenes permitidos y validar endpoints con pruebas HTTP
Falta de responsividad visual	Media	Medio	Probar Angular en diferentes tamanos de pantalla y usar componentes flexibles
Perdida o corrupcion de datos locales	Baja	Medio	Mantener scripts de generacion de datos y repositorio SQLite recreable

3. Analisis de la Situacion Actual

3.1. Planteamiento del problema

Actualmente, la informacion relacionada con la presencia digital de estudiantes en redes sociales puede encontrarse dispersa en distintas plataformas. Esto dificulta obtener una vision consolidada sobre niveles de participacion, alcance de publicaciones, rendimiento por red social o comparacion entre ciclos academicos.

Para una escuela profesional, contar con indicadores de presencia digital puede aportar valor en actividades de seguimiento academico, visibilidad institucional, orientacion profesional y toma de decisiones sobre estrategias de comunicacion. Sin embargo, la consulta manual de publicaciones o interacciones resulta poco practica, repetitiva y dificil de analizar.

El proyecto EPIS Analytics propone un prototipo que centraliza datos simulados de publicaciones de estudiantes y los presenta mediante dashboards. Aunque los datos son simulados, el diseno permite representar un escenario realista y deja abierta la posibilidad de integrar fuentes externas o APIs oficiales en una etapa posterior.

3.2. Consideraciones de hardware y software

Hardware para desarrollo

Recurso	Descripcion	Estado
Laptop o PC de desarrollo	Equipo con Windows, Linux o macOS	Disponible
Memoria RAM	Minimo 8 GB recomendado	Disponible
Procesador	Intel/AMD moderno o equivalente	Disponible
Almacenamiento	Minimo 5 GB para codigo, dependencias y base local	Disponible
Conexion a internet	Necesaria para dependencias, GitHub y posible despliegue cloud	Disponible

Software para desarrollo

Software	Uso
Python	Backend, servicios, pruebas y generacion de datos
FastAPI	API REST del sistema
SQLite	Base de datos local del prototipo
Angular	Frontend web
Node.js y npm	Ejecucion y compilacion del frontend
ECharts	Visualizacion grafica
Pytest	Pruebas unitarias y de API

Software	Uso
Terraform	Definicion de infraestructura como codigo
Infracost	Estimacion de costos cloud a partir de Terraform
GitHub	Control de versiones y documentacion del proyecto

Arquitectura actual

El sistema se organiza como una arquitectura cliente-servidor:

Capa	Tecnologia	Responsabilidad
Frontend	Angular	Login, vistas, filtros, graficos y consumo HTTP
API	FastAPI	Endpoints REST, CORS, autenticacion y respuestas JSON
Servicios	Python	Logica de negocio, analitica y autenticacion
Repositorios	Python + SQLite	Acceso y persistencia de datos
Modelos	Python dataclasses	Representacion de entidades como Publicacion y Usuario
Infraestructura propuesta	Terraform + AWS	Despliegue controlado y estimacion de costos

4. Estudio de Factibilidad

El estudio de factibilidad evalua si el sistema EPIS Analytics puede desarrollarse e implantarse de manera tecnica, economica, operativa, legal, social y ambiental. Para la evaluacion se considera el prototipo actualmente implementado y una propuesta de despliegue cloud de bajo costo declarada con Terraform.

4.1. Factibilidad Tecnica

El proyecto es tecnicamente factible debido a que utiliza herramientas ampliamente disponibles, documentadas y compatibles entre si.

Evaluacion tecnica

Elemento	Evaluacion
Frontend Angular	Permite construir una interfaz web modular, responsive y separada del backend
Backend FastAPI	Permite exponer endpoints REST de forma rapida, clara y testeable
SQLite	Suficiente para prototipo academico y datos simulados
Repositorios	Facilitan cambiar SQLite por PostgreSQL u otra base en el futuro
Login con token	Permite proteger rutas sin depender todavia de OAuth externo
ECharts	Mejora la visualizacion de datos con graficos reales
Terraform	Permite reproducir infraestructura y evitar configuracion manual
Infracost	Permite estimar costos antes de desplegar infraestructura

Infraestructura propuesta con Terraform

Para el prototipo desplegable se propone una infraestructura minima en AWS. La definicion completa se encuentra en el directorio `infra/terraform`, compuesto por:

Archivo	Proposito
`main.tf`	Define proveedor AWS, EC2, Security Group, S3, CloudFront y politicas
`variables.tf`	Declara variables parametrizables del despliegue
`outputs.tf`	Expone URL de API, IP publica y dominio CloudFront
`terraform.tfvars.example`	Muestra valores de ejemplo para ejecutar el plan
`user_data.sh.tftpl`	Configura dependencias base y Nginx en la instancia EC2
`README.md`	Documenta comandos de validacion, costos y advertencias

La infraestructura propuesta contiene:

Recurso Terraform	Servicio cloud	Proposito
`aws_instance`	Amazon EC2 t3.micro	Ejecutar backend FastAPI y servir API
`aws_ebs_volume` o volumen raiz gp3	Amazon EBS gp3	Almacenar sistema, app y base SQLite del prototipo
`aws_s3_bucket`	Amazon S3	Publicar archivos estaticos generados por Angular
`aws_cloudfront_distribution`	Amazon CloudFront	Distribuir frontend con cache y HTTPS
`aws_security_group`	Security Group	Controlar acceso HTTP/HTTPS/SSH
`aws_iam_role`	IAM	Permisos minimos para operacion del servidor

Extracto de la estructura Terraform incluida:

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_instance" "api" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  root_block_device {
    volume_type = "gp3"
    volume_size = 20
  }

  tags = {
    Project = "epis-analytics"
    Env     = "dev"
  }
}

resource "aws_s3_bucket" "frontend" {
  bucket = var.frontend_bucket_name

  tags = {
```

```
    Project = "epis-analytics"
    Env     = "dev"
  }
}
```

Comandos propuestos desde `infra/terraform`:

```
terraform init
terraform fmt
terraform validate
terraform plan -out=tfplan
infracost breakdown --path . --terraform-var frontend_bucket_name=epis-
         analytics-dev-frontend-demo
```

El uso de Terraform es factible porque permite definir la infraestructura en archivos versionables, revisables y reutilizables. De acuerdo con la documentacion oficial de HashiCorp, Terraform permite construir, cambiar y versionar infraestructura de forma segura y eficiente mediante infraestructura como codigo.

No se requiere desplegar la infraestructura para cumplir el analisis de factibilidad; sin embargo, el modulo queda preparado para ejecutar `terraform init`, `terraform validate`, `terraform plan` e `infracost breakdown`. El comando `terraform apply` solo debe ejecutarse si se cuenta con credenciales AWS, presupuesto aprobado y revision previa de seguridad.

Validacion local realizada:

Herramienta	Version	Resultado
Terraform	1.14.9	`terraform init` y `terraform validate` ejecutados correctamente
Infracost	0.10.44	`infracost breakdown --path .` ejecutado correctamente

4.2. Factibilidad Economica

El proyecto es economicamente factible porque el desarrollo utiliza principalmente herramientas libres o de uso gratuito, y la infraestructura propuesta se mantiene en un nivel minimo para prototipo.

4.2.1. Costos generales

Concepto	Cantidad	Costo unitario estimado	Subtotal
Material de oficina	1	S/ 30.00	S/ 30.00
Energia electrica referencial	1	S/ 50.00	S/ 50.00
Internet referencial	1	S/ 80.00	S/ 80.00
Impresiones/documentacion	1	S/ 20.00	S/ 20.00
Total costos generales			**S/ 180.00**

4.2.2. Costos operativos durante el desarrollo

Concepto	Costo estimado
Uso de equipos propios	S/ 0.00
Software de desarrollo libre	S/ 0.00
Repositorio GitHub academico	S/ 0.00
Herramientas de pruebas locales	S/ 0.00
Total costos operativos	**S/ 0.00**

4.2.3. Costos del ambiente

Recurso	Descripcion	Costo estimado
Python/FastAPI	Framework backend	S/ 0.00
Angular	Framework frontend	S/ 0.00
SQLite	Base de datos local	S/ 0.00
Terraform	Infraestructura como codigo	S/ 0.00
Infracost CLI	Estimacion de costos para Terraform	S/ 0.00 para uso base
AWS	Ambiente cloud propuesto	Variable segun consumo
Total base del ambiente	Sin considerar consumo cloud	**S/ 0.00**

4.2.4. Costos de personal

Los costos de personal se estiman con fines academicos y no implican contratacion real.

Rol	Horas estimadas	Tarifa referencial	Subtotal
Analista / jefe de proyecto	40	S/ 25.00	S/ 1,000.00
Desarrollador backend	50	S/ 25.00	S/ 1,250.00
Desarrollador frontend	50	S/ 25.00	S/ 1,250.00
QA / documentacion	30	S/ 20.00	S/ 600.00
Total costos de personal	**170**		**S/ 4,100.00**

4.2.5. Costos totales del desarrollo del sistema

Categoria	Total
Costos generales	S/ 180.00
Costos operativos	S/ 0.00
Costos del ambiente base	S/ 0.00
Costos de personal	S/ 4,100.00
Costo total estimado de desarrollo	**S/ 4,280.00**

4.2.6. Analisis de costos cloud con Terraform

Para el analisis de costos cloud se considera una arquitectura minima de despliegue declarada con Terraform. El objetivo no es desplegar una infraestructura de alta disponibilidad, sino estimar un ambiente viable para prototipo academico.

Supuestos de calculo

Supuesto	Valor
Proveedor cloud	AWS
Region	US East (N. Virginia)
Backend	1 instancia EC2 t3.micro Linux
Almacenamiento backend	20 GB EBS gp3
Frontend	Angular compilado en S3
CDN	CloudFront plan gratuito o pay-as-you-go de bajo consumo
Trafico mensual estimado	Menor a 100 GB
Base de datos	SQLite para prototipo
Tipo de cambio referencial	1 USD = S/ 3.75

Estimacion mensual de infraestructura

Recurso Terraform	Servicio	Formula	Costo mensual estimado
`aws_instance.api`	EC2 t3.micro	USD 0.0104/hora x 730 horas	USD 7.59
`root_block_device`	EBS gp3 20 GB	20 GB x USD 0.08/GB-mes	USD 1.60
`aws_s3_bucket.frontend`	S3 Standard	Depende de almacenamiento y solicitudes	Variable
`aws_cloudfront_distribution.frontend`	CloudFront	Depende de transferencia y solicitudes	Variable
Transferencia de datos	AWS data transfer	Menor a 100 GB/mes segun supuesto academico	Variable
Total base mensual estimado por Infracost			**USD 9.19**

Equivalente aproximado en soles:

```
USD 9.19 x S/ 3.75 = S/ 34.46 mensuales
```

Proyeccion anual:

Periodo	Costo USD	Costo aproximado en soles
1 mes	USD 9.19	S/ 34.46
6 meses	USD 55.14	S/ 206.78
12 meses	USD 110.28	S/ 413.55

Resultado esperado con Infracost

El costo puede validarse antes del despliegue con:

```
infracost breakdown --path infra/terraform
```

Salida esperada de manera referencial:

```
Project: epis-analytics

Name                                        Quantity       Unit        Monthly
Cost
aws_instance.api
  Instance usage (Linux/UNIX, t3.micro)     730            hours       $7.59
  root_block_device (gp3, 20GB)             20             GB-month    $1.60

aws_s3_bucket.frontend
  Storage (S3 Standard)                     1              GB-month    $0.02

aws_cloudfront_distribution.frontend
  Data transfer                             low usage      GB          $0.00

OVERALL TOTAL                                                          $9.19
```

Resultado real obtenido en la validacion local:

```
Terraform v1.14.9
Infracost v0.10.44
terraform init: correcto
terraform validate: Success! The configuration is valid.
infracost breakdown --path . --terraform-var frontend_bucket_name=epis-
analytics-dev-frontend-demo: OVERALL TOTAL $9.19
```

Interpretacion del costo

La estimacion demuestra que el proyecto puede operar en un ambiente cloud basico con un costo mensual bajo. La decision de usar una instancia t3.micro y almacenamiento gp3 permite mantener el gasto controlado. CloudFront y S3 reducen carga sobre el backend al servir el frontend como contenido estatico.

Para una version productiva se recomienda reevaluar el uso de SQLite y considerar PostgreSQL gestionado o un contenedor con base de datos separada. Esa mejora incrementaria el costo, pero aumentaria disponibilidad, seguridad y escalabilidad.

Medidas de control de costos

Medida	Beneficio
Usar Terraform para crear y destruir ambientes	Evita recursos olvidados
Ejecutar `terraform plan` antes de aplicar cambios	Permite revisar recursos nuevos
Ejecutar `infracost breakdown` antes de desplegar	Permite estimar costos mensuales
Etiquetar recursos con `Project` y `Env`	Facilita seguimiento de costos
Evitar NAT Gateway en prototipo	Reduce costos fijos innecesarios
Usar una sola instancia pequena	Mantiene bajo el gasto mensual
Mantener S3 y CloudFront para frontend	Reduce carga del servidor backend
Apagar o destruir ambiente cuando no se use	Reduce costos de computo

4.3. Factibilidad Operativa

El sistema es operativamente factible porque su uso principal es sencillo: el usuario inicia sesion, ingresa al dashboard personal o global, selecciona filtros y visualiza indicadores.

Usuarios interesados

Interesado	Necesidad
Estudiantes EPIS	Consultar sus metricas personales de contenido social
Docentes	Observar tendencias generales de participacion
Coordinacion academica	Analizar indicadores por ciclo y red social
Equipo de desarrollo	Mantener, probar y extender el sistema

Beneficios operativos

Beneficio	Descripcion
Centralizacion	La informacion se consulta desde una unica interfaz
Rapidez	Los indicadores se calculan automaticamente
Comparacion	Permite observar redes sociales, ciclos y estudiantes
Seguridad inicial	Los dashboards se protegen con login y token
Escalabilidad funcional	La arquitectura permite agregar nuevos filtros o fuentes de datos

4.4. Factibilidad Legal

El proyecto es legalmente factible en su etapa actual porque trabaja con datos simulados, no consume informacion real de usuarios ni extrae datos privados de redes sociales.

Consideraciones legales:

Aspecto	Evaluacion
Datos personales	No se usan datos reales sensibles en el prototipo
Redes sociales	No se realiza scraping ni consumo no autorizado de APIs
Autenticacion	Se utiliza login local con contrasena hasheada
Licencias	Las herramientas usadas son de uso libre o abierto para desarrollo
Futuras integraciones	Deberan respetar terminos de uso de LinkedIn, Instagram, YouTube y normativa de proteccion de datos

Para una version con datos reales se recomienda obtener consentimiento informado, definir politicas de privacidad y cumplir la Ley de Proteccion de Datos Personales aplicable en Peru.

4.5. Factibilidad Social

El sistema es socialmente factible porque puede aportar informacion util para mejorar la visibilidad academica y profesional de estudiantes. Su uso puede apoyar estrategias de comunicacion, orientacion y seguimiento institucional.

Se debe evitar que las metricas se utilicen como mecanismo de comparacion negativa entre estudiantes. Los rankings y graficos deben interpretarse como indicadores de actividad digital, no como evaluacion integral del desempeno academico o personal.

4.6. Factibilidad Ambiental

El impacto ambiental del proyecto es bajo, ya que el sistema se desarrolla y ejecuta principalmente en medios digitales. No requiere infraestructura fisica adicional significativa ni consumo de materiales en gran escala.

Medidas ambientales recomendadas:

Medida	Impacto
Documentacion digital	Reduce uso de papel
Infraestructura cloud minima	Evita uso de servidores fisicos propios
Apagado/destruccion de ambientes no usados	Reduce consumo energetico indirecto
Uso de recursos pequenos	Reduce huella operacional del prototipo

5. Analisis Financiero

El analisis financiero evalua si el beneficio del proyecto justifica los costos estimados de desarrollo e infraestructura.

5.1. Justificacion de la inversion

EPIS Analytics se justifica porque permite transformar datos dispersos o simulados de actividad social en informacion visual y consultable. Aunque el prototipo no reemplaza sistemas institucionales existentes, demuestra una solucion viable para analitica academica y puede evolucionar hacia integraciones reales.

5.1.1. Beneficios del proyecto

Tipo de beneficio	Descripcion
Tangible	Reduccion de tiempo para generar reportes o revisar indicadores manualmente
Tangible	Disponibilidad de dashboards para consulta inmediata
Tangible	Base tecnica reutilizable para futuros modulos
Intangible	Mejora en la toma de decisiones academicas
Intangible	Mayor visibilidad de la actividad digital estudiantil
Intangible	Experiencia practica en arquitectura moderna, API REST, frontend y Terraform

5.1.2. Criterios de inversion

5.1.2.1. Relacion beneficio/costo

Para una evaluacion academica, se asigna un beneficio estimado conservador equivalente a S/ 6,000.00 por la utilidad del sistema, aprendizaje tecnico, documentacion, reutilizacion del codigo y posibilidad de evolucion.

```
Beneficio estimado = S/ 6,000.00
Costo de desarrollo = S/ 4,280.00
Relacion B/C = 6,000.00 / 4,280.00 = 1.40
```

Como la relacion beneficio/costo es mayor a 1, el proyecto se considera economicamente aceptable.

5.1.2.2. Valor Actual Neto

Para el prototipo academico no existe flujo real de ingresos. Sin embargo, puede realizarse una aproximacion considerando beneficios internos proyectados.

Supuestos:

Concepto	Valor
Inversion inicial	S/ 4,280.00
Beneficio estimado anual	S/ 6,000.00
Costo cloud anual estimado	S/ 414.45
Beneficio neto anual	S/ 5,585.55

```
VAN aproximado = Beneficio neto anual - Inversion inicial
VAN aproximado = S/ 5,585.55 - S/ 4,280.00
VAN aproximado = S/ 1,305.55
```

El VAN aproximado es positivo, por lo que el proyecto resulta factible bajo los supuestos planteados.

5.1.2.3. Tasa Interna de Retorno

Al tratarse de un proyecto academico con un solo periodo de evaluacion, la TIR se estima de forma simplificada:

```
TIR aproximada = (Beneficio neto - Inversion inicial) / Inversion inicial
TIR aproximada = (5,585.55 - 4,280.00) / 4,280.00
TIR aproximada = 30.50%
```

La tasa aproximada es positiva, por lo que el proyecto puede considerarse financieramente viable para un escenario academico y de prototipo.

6. Conclusiones

1. El proyecto EPIS Analytics es tecnicamente factible porque utiliza tecnologias modernas, disponibles y compatibles: Angular, FastAPI, SQLite, ECharts, Pytest y Terraform.

2. La arquitectura cliente-servidor permite separar claramente la interfaz web del backend, facilitando mantenimiento, pruebas y futuras ampliaciones.

3. La incorporacion de login real con usuario, contrasena, hash y token firmado mejora la seguridad del prototipo frente al login simulado inicial.

4. El proyecto es economicamente viable porque los costos de desarrollo son moderados y la infraestructura cloud minima estimada con Terraform se mantiene alrededor de USD 9.19 mensuales como costo base bajo los supuestos definidos.

5. El uso de Terraform e Infracost permite controlar el riesgo de costos cloud, ya que la infraestructura puede revisarse, estimarse y versionarse antes del despliegue.

6. La factibilidad legal es favorable mientras se mantengan datos simulados. Si se incorporan datos reales de redes sociales, sera necesario revisar permisos, consentimiento y terminos de uso de cada plataforma.

7. La factibilidad operativa es positiva porque el sistema ofrece una experiencia clara: login, dashboard personal, dashboard global, filtros por red social y visualizacion de metricas.

8. Se concluye que el proyecto es viable y factible para continuar su desarrollo como prototipo academico, con posibilidad de evolucion hacia una version productiva.

7. Referencias

- HashiCorp Developer. *Terraform overview*. https://developer.hashicorp.com/terraform/docs
- HashiCorp Developer. *What is Terraform*. https://developer.hashicorp.com/terraform/intro
- Infracost. *Get started*. https://www.infracost.io/docs/
- AWS. *Amazon EC2 T3 Instances*. https://aws.amazon.com/ec2/instance-types/t3/
- AWS. *Amazon EBS General Purpose Volumes*. https://aws.amazon.com/ebs/general-purpose/
- AWS. *Amazon S3 Pricing*. https://aws.amazon.com/s3/pricing/
- AWS. *Amazon CloudFront Pricing*. https://aws.amazon.com/cloudfront/pricing/